



University
of Exeter

COMM113

Deep Learning – Week 3

Regularisations I

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Review

Deep Learning

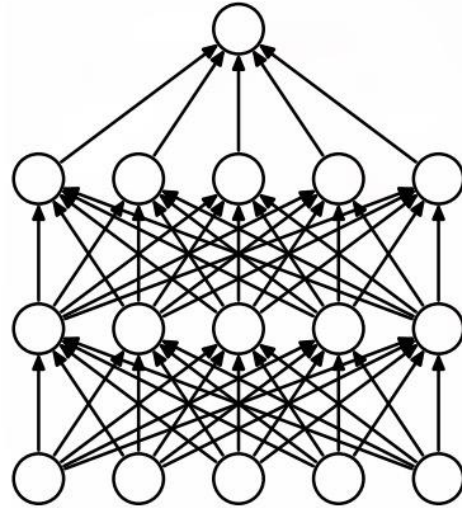
- [Table of Contents](#)
- [Acknowledgements](#)
- [Notation](#)
- [1 Introduction](#)
- [Part I: Applied Math and Machine Learning Basics](#)
 - [2 Linear Algebra](#)
 - [3 Probability and Information Theory](#)
 - [4 Numerical Computation](#)
 - [5 Machine Learning Basics](#)
- [Part II: Modern Practical Deep Networks](#)
 - [6 Deep Feedforward Networks](#)
 - [7 Regularization for Deep Learning](#)
 - [8 Optimization for Training Deep Models](#)
 - [9 Convolutional Networks](#)
 - [10 Sequence Modeling: Recurrent and Recursive Nets](#)
 - [11 Practical Methodology](#)
 - [12 Applications](#)
- [Part III: Deep Learning Research](#)
 - [13 Linear Factor Models](#)
 - [14 Autoencoders](#)
 - [15 Representation Learning](#)
 - [16 Structured Probabilistic Models for Deep Learning](#)
 - [17 Monte Carlo Methods](#)
 - [18 Confronting the Partition Function](#)
 - [19 Approximate Inference](#)
 - [20 Deep Generative Models](#)

Please read Part 1 of the Textbook "Deep Learning"

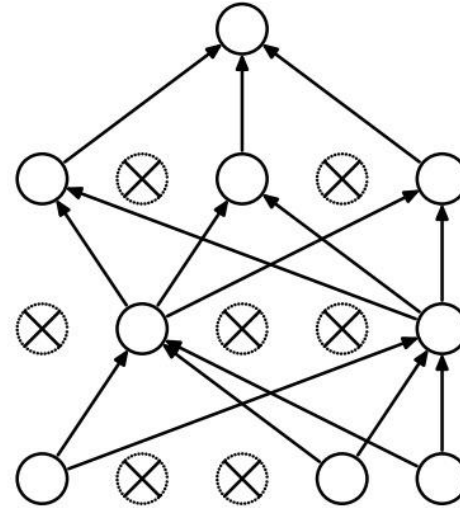
[Deep Learning](https://www.deeplearningbook.org/)

<https://www.deeplearningbook.org/>

Dropout



(a) Standard Neural Net



(b) After applying dropout.

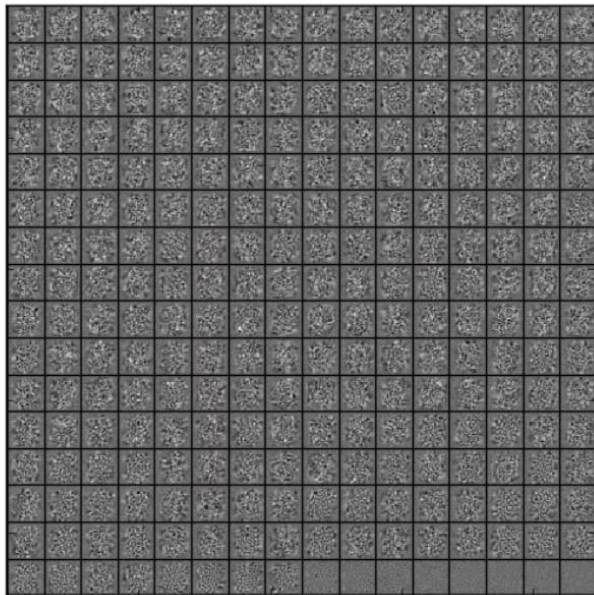
At each training iteration,
With probability p to
“dropout”

p : dropout rate

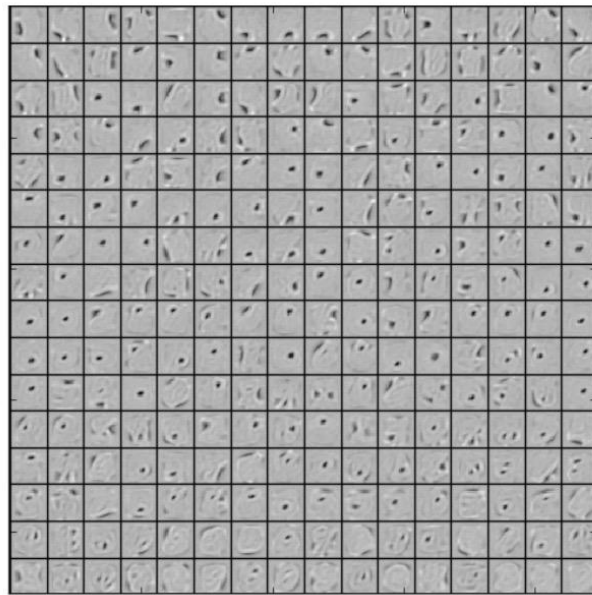
Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

A first “deep” regularization technique is dropout (Srivastava et al., 2014). It consists of removing units at random during the forward pass on each sample, and putting them all back during test.

Dropout



(a) Without dropout



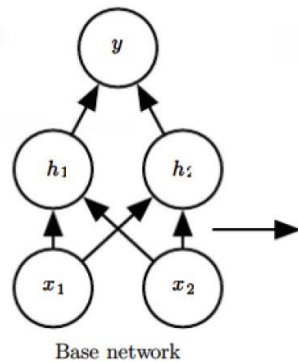
(b) Dropout with $p = 0.5$.

Figure 7: Features learned on MNIST with one hidden layer autoencoders having 256 rectified linear units.

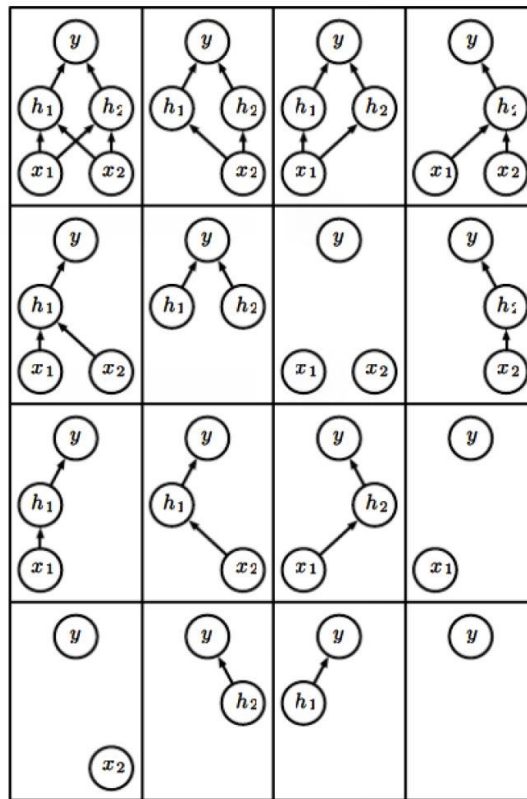
A network with dropout can be interpreted as an ensemble of many networks (shared weights).

Dropout as an ensemble

- Remove non-output units from base network.
- Remaining 4 units yield 16 networks



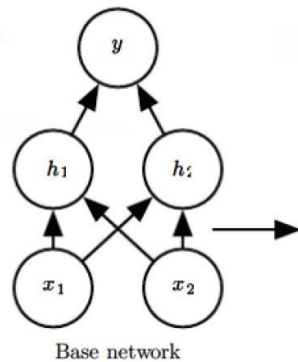
- Here many networks have no path from input to output
- Problem insignificant with large networks



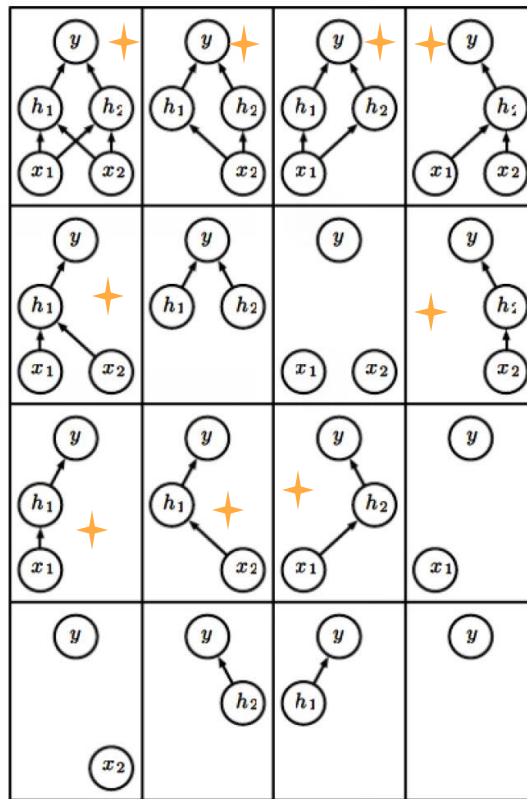
Which subnets can be updated?

Dropout as an ensemble

- Remove non-output units from base network.
- Remaining 4 units yield 16 networks

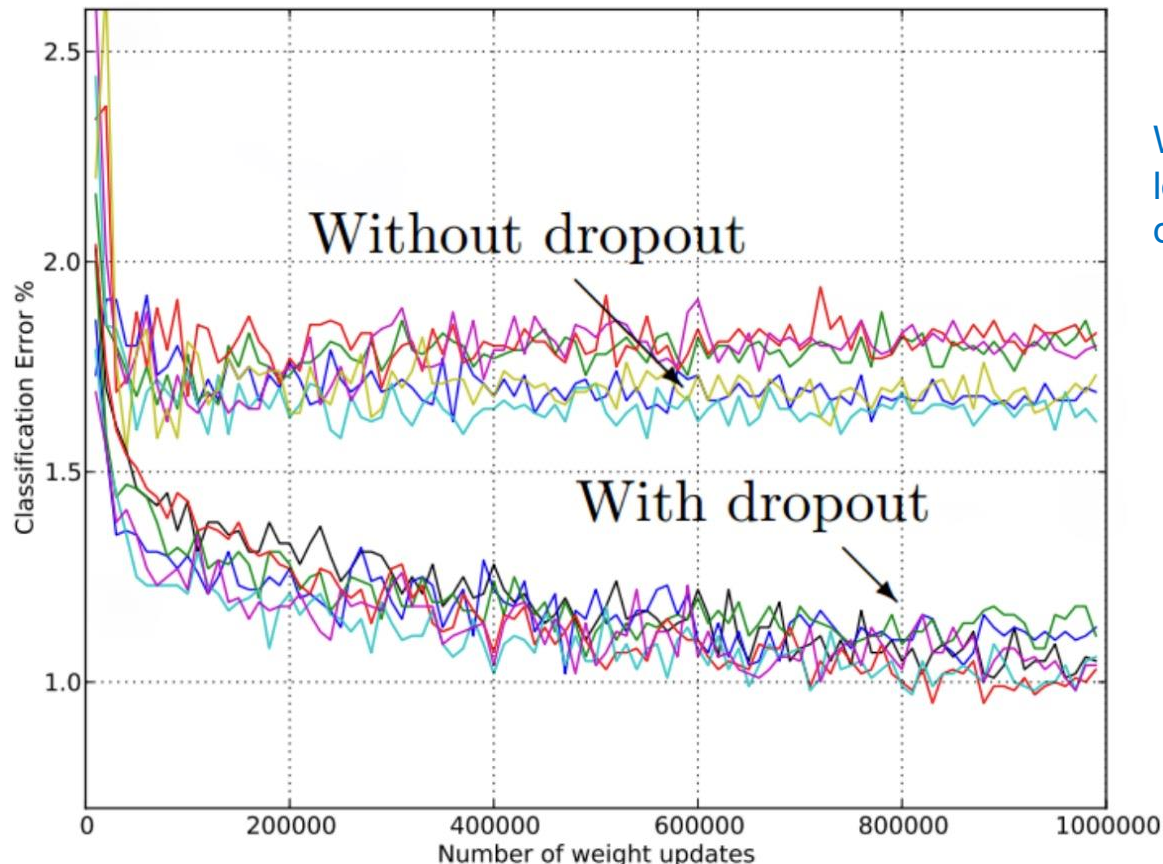


- Here many networks have no path from input to output
- Problem insignificant with large networks



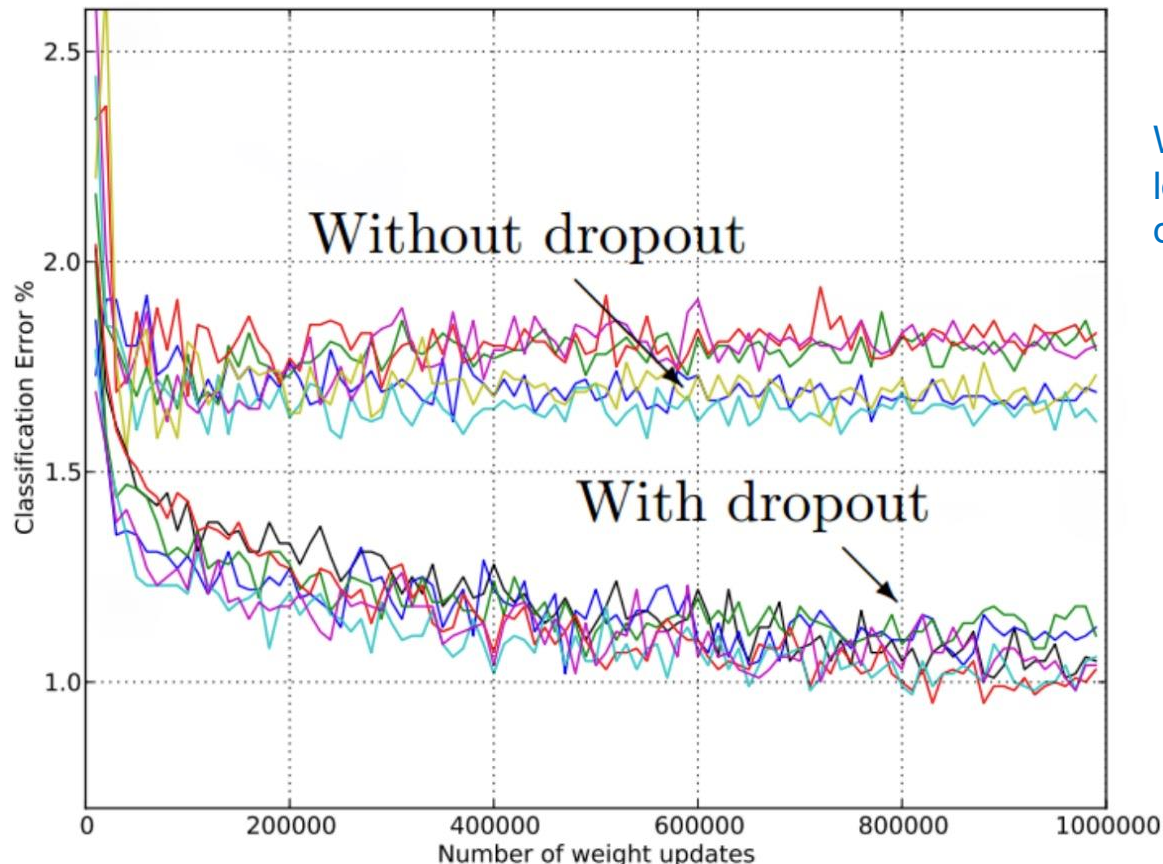
Which subnets can be updated?

Performance with/without Dropout



What would you expect the training loss curve to look like with vs without dropout?

Performance with/without Dropout



What would you expect the training loss curve to look like with vs without dropout?

With dropout: slower convergence but lower test error.

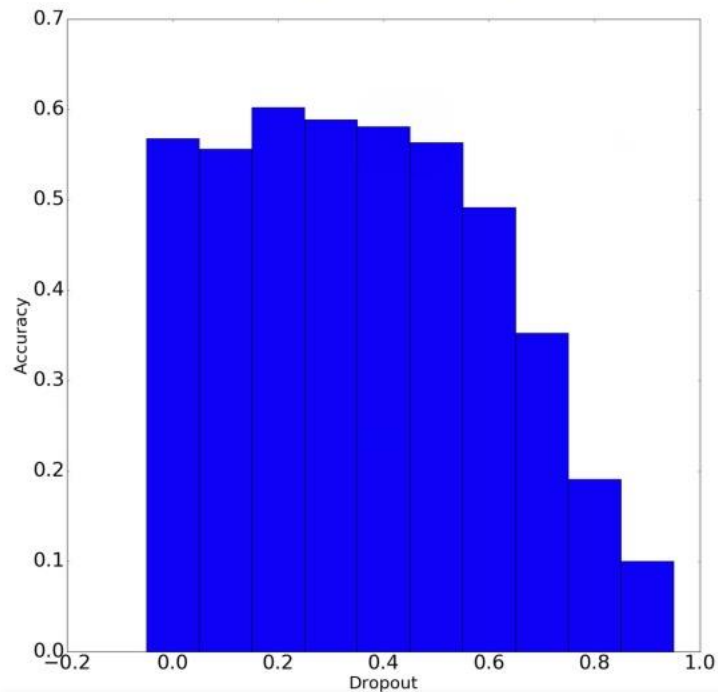
Dependence on dropout rate p

Higher p , better performance? What happens if the dropout rate p is too high?

Dependence on dropout rate p

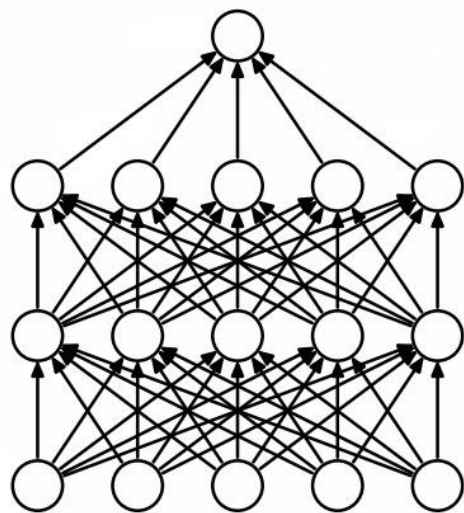
Higher p , better performance? What happens if the dropout rate p is too high?

Accuracy vs dropout

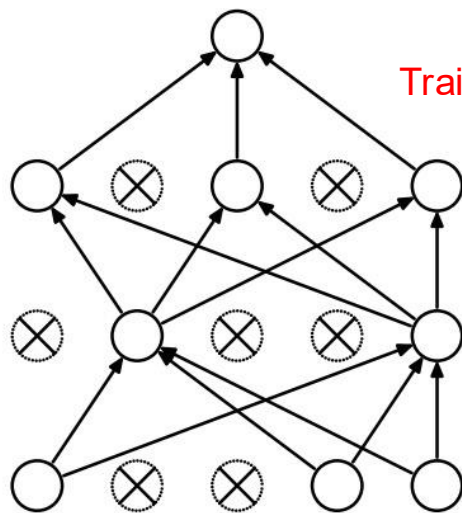


The network underfits because too much information is lost.

Dropout



(a) Standard Neural Net



(b) After applying dropout.

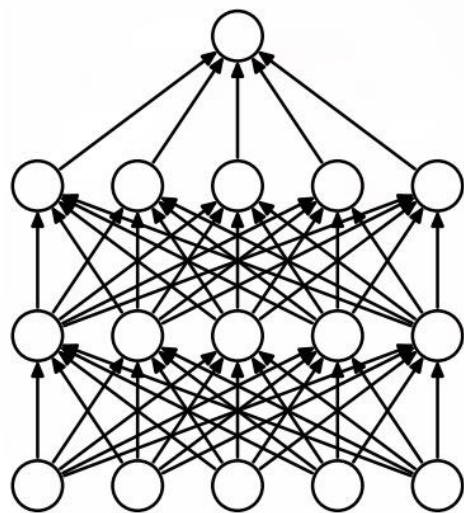
Training time: dropout with rate p

Test time: no dropout

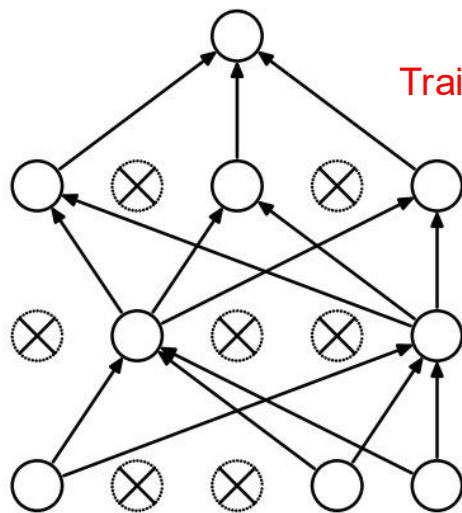
Why not keep dropout active during testing?

Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Dropout



(a) Standard Neural Net



(b) After applying dropout.

Training time: dropout with rate p

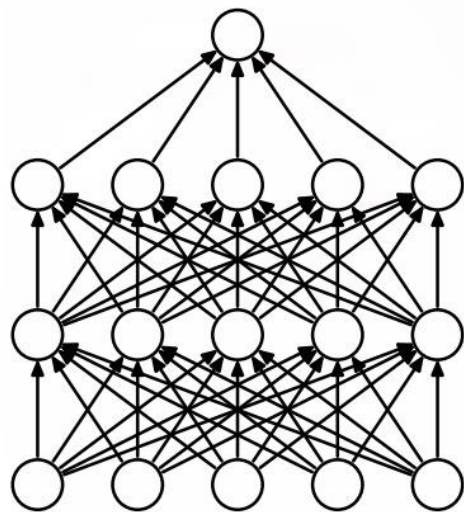
Test time: no dropout

Why not keep dropout active during testing?

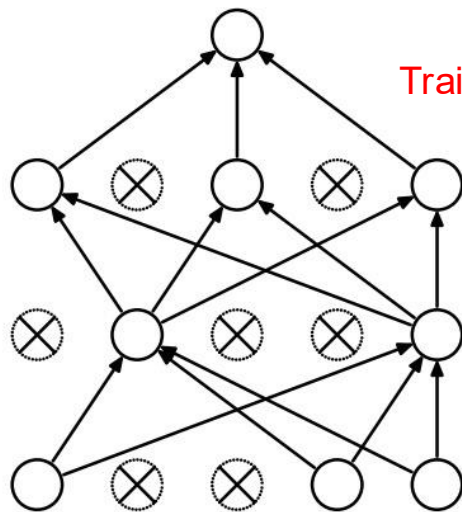
Because we want deterministic predictions, not random ones.

Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Dropout



(a) Standard Neural Net



(b) After applying dropout.

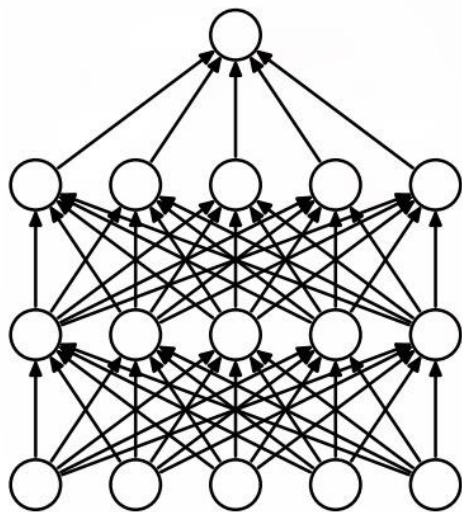
Training time: dropout with rate p

Test time: no dropout

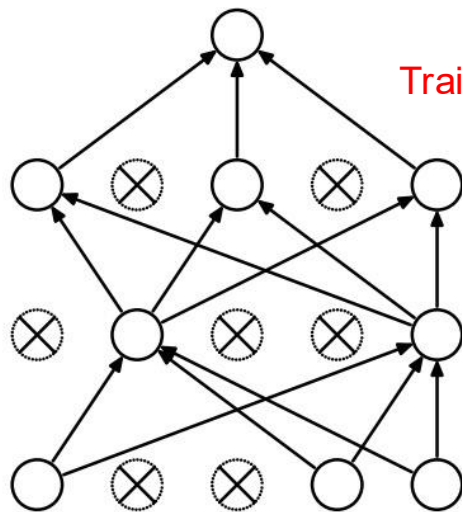
What's the problem here?

Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Dropout



(a) Standard Neural Net



(b) After applying dropout.

Training time: dropout with rate p

Test time: no dropout

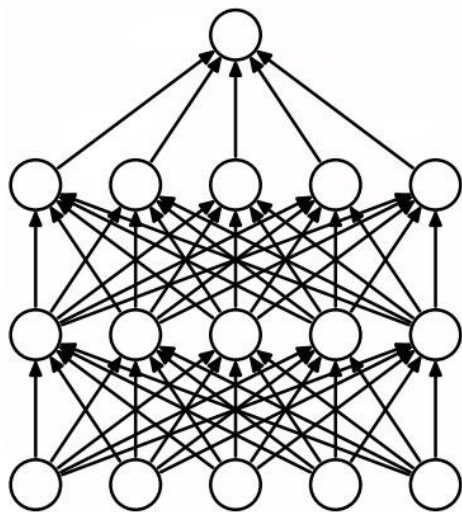
What's the problem here?

Test without dropout: the expectation of a neuron $E(x)$

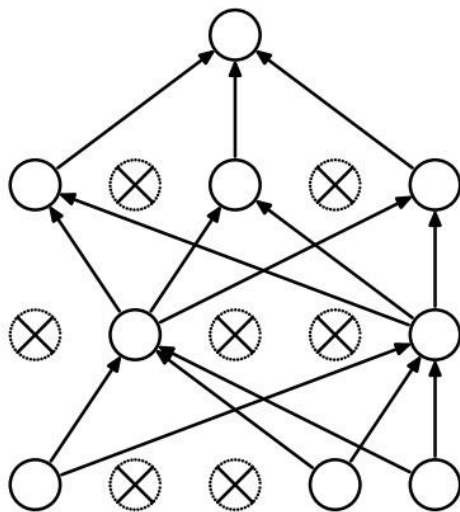
Train with dropout: the expectation of a neuron $E((1-p)x)$

Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Dropout



(a) Standard Neural Net



(b) After applying dropout.

What's the problem here?

Test without dropout: the expectation of a neuron $E(x)$

Train with dropout: the expectation of a neuron $E((1-p)x)$

To keep same expectation, multiplying activations by $1/(1-p)$ during training



Train expectation $E(x)$

Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Dropout - PyTorch

```
# Using PyTorch's built-in Dropout  
model = nn.Sequential(  
    nn.Linear(10, 20),  
    nn.Dropout(p=0.5), # 50% dropout probability  
    nn.ReLU(),  
    nn.Linear(20, 2)  
)
```

In PyTorch, what happens if you forget to call `model.eval()` before testing?

Dropout - PyTorch

Using PyTorch's built-in Dropout

```
model = nn.Sequential(  
    nn.Linear(10, 20),  
    nn.Dropout(p=0.5),    # 50% dropout probability  
    nn.ReLU(),  
    nn.Linear(20, 2)  
)
```

In PyTorch, what happens if you forget to call `model.eval()` before testing?

Dropout remains active — your predictions will be inconsistent.

Dropout - Numpy

```
def dropout_pytorch(x, p=0.5, training=True):  
    if not training:  
        return x
```

```
# Create mask: 1 with probability (1-p), 0 with probability p  
mask = (torch.rand_like(x) > p).float()
```

```
# Scale by 1/(1-p) to maintain expected value  
return x * mask / (1 - p)
```

Data Augmentation

Expand the size of a training set by creating modified data from the existing one.



Data Augmentation



Original Image



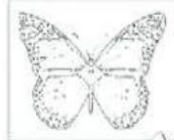
De-texturized



De-colored



Edge Enhanced



Salient Edge Map



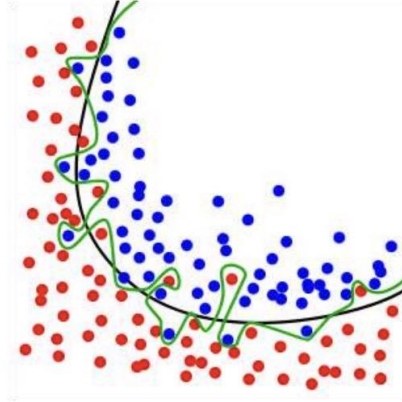
Flip/Rotate

Imagine teaching a model to recognise a cat — showing the same cat from different angles helps it learn.

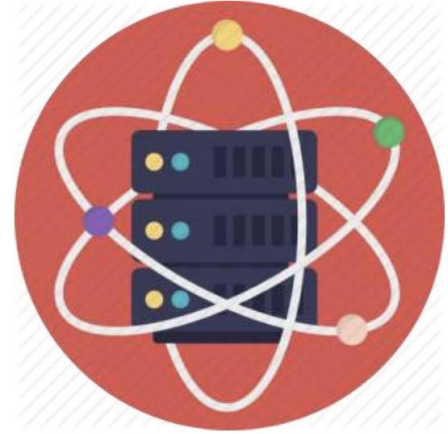
Purpose of Data Augmentation



Enlarge
dataset



Prevent
overfitting



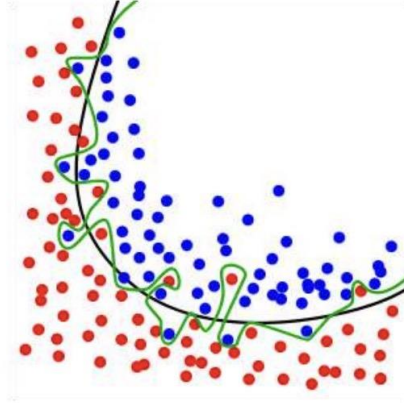
Improve
performance model

Why does data augmentation reduce overfitting?

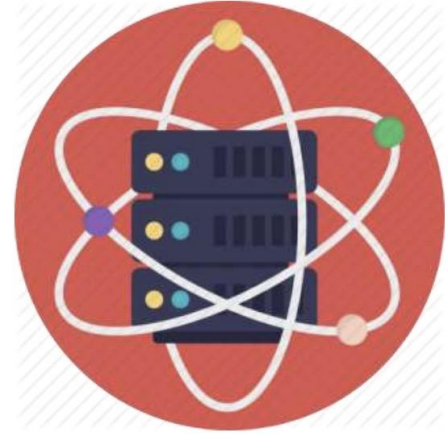
Purpose of Data Augmentation



Enlarge
dataset



Prevent
overfitting



Improve
performance model

Why does data augmentation reduce overfitting?

It makes the model invariant to small perturbations

What to Augment



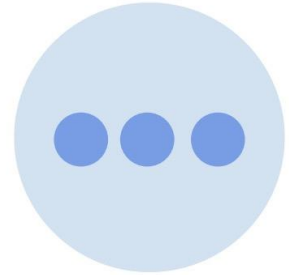
Audio



Texts

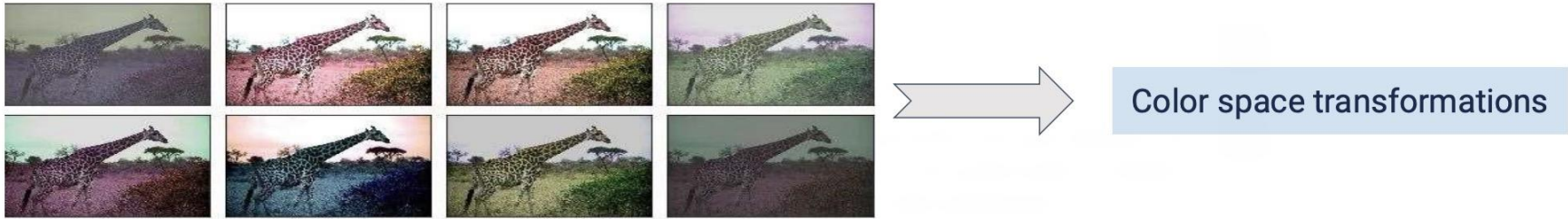
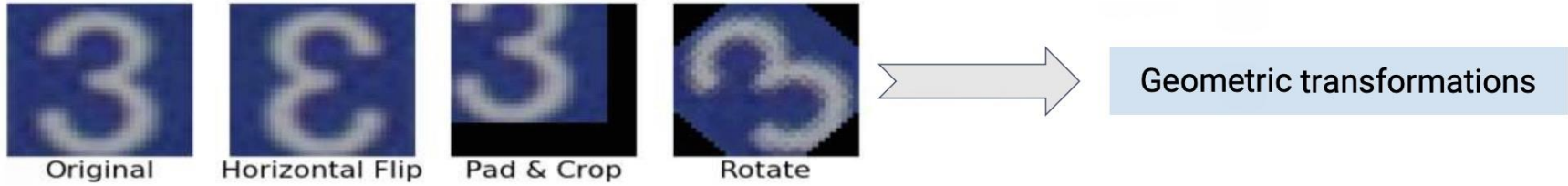


Images



Any other
types

Data Augmentation techniques : For Images



If you rotate a '6' by 180° , what digit might it become?
A '9'. So not all transformations are valid!

Data Augmentation techniques : For Text

In my presentation focus topic is Data Augmentation.

In my presentation **theme** topic is Data Augmentation – Synonym Replacement

In my presentation focus topic is Data Augmentation **techniques** – Random Insertion

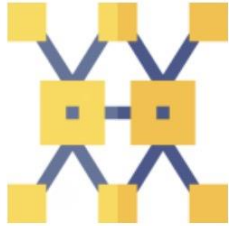
In my presentation **topic focus** is Data Augmentation -- Random Swap

In my presentation focus topic Data Augmentation --Random Deletion

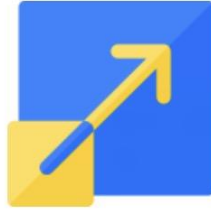
When do I use data augmentation?



small data set



complex problem



transformation of
data would be
effective and
easy



You know how to
find the correct
method for your
data



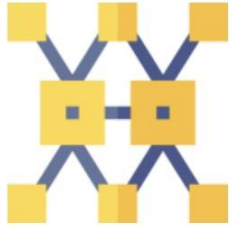
You have a
strategy for
dealing with
overfitting

Can augmentation ever *hurt* performance?

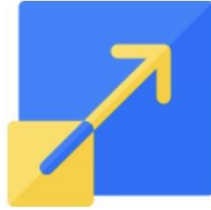
When do I use data augmentation?



small data set



complex problem



transformation of
data would be
effective and
easy



You know how to
find the correct
method for your
data



You have a
strategy for
dealing with
overfitting

Can augmentation ever *hurt* performance?

Yes, if the transformation distorts the label or adds too much noise.

Data augmentation in PyTorch

```
import torchvision.transforms as transforms
from PIL import Image
import numpy as np

# Using PyTorch's built-in transforms
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.RandomCrop(size=(224, 224)),
    transforms.ToTensor(),
])
```

Data augmentation in Numpy

```
# Random horizontal flip
if np.random.rand() < flip_prob:
    aug_image = np.fliplr(aug_image)

# Random vertical flip
if np.random.rand() < flip_prob:
    aug_image = np.flipud(aug_image)

# Random rotation (90-degree increments)
k = np.random.randint(0, 4) # 0, 1, 2, or 3 for 0°, 90°, 180°, 270°
aug_image = np.rot90(aug_image, k=k)

# Add random noise
noise = np.random.normal(0, 0.01, aug_image.shape)
aug_image = np.clip(aug_image + noise, 0, 1)

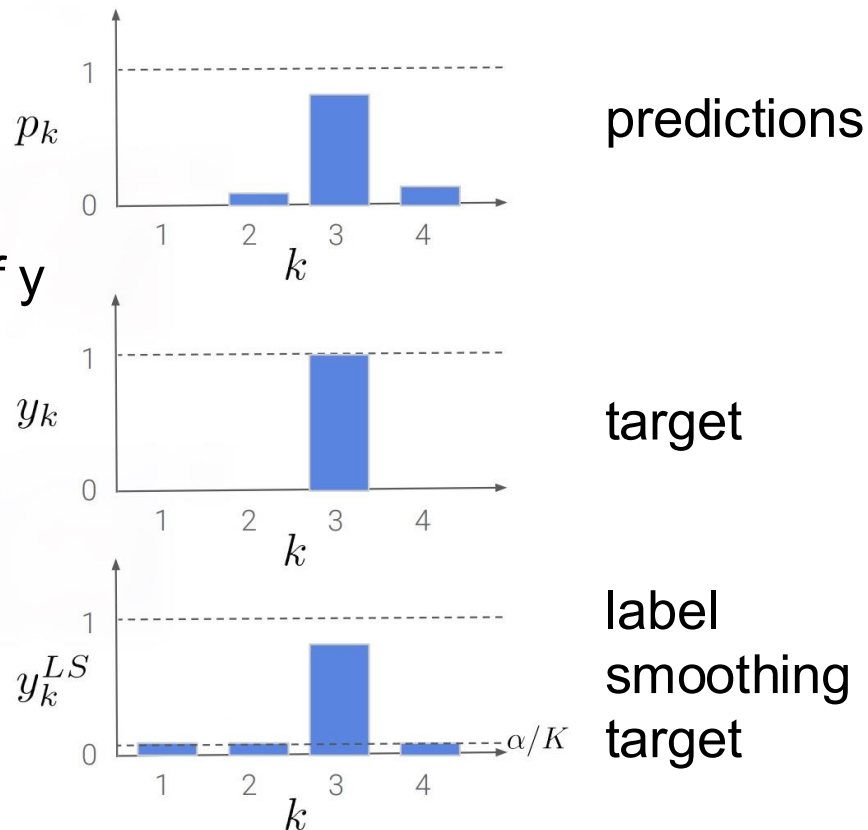
# Random brightness adjustment
brightness_factor = np.random.uniform(0.8, 1.2)
aug_image = np.clip(aug_image * brightness_factor, 0, 1)
```

Label smoothing

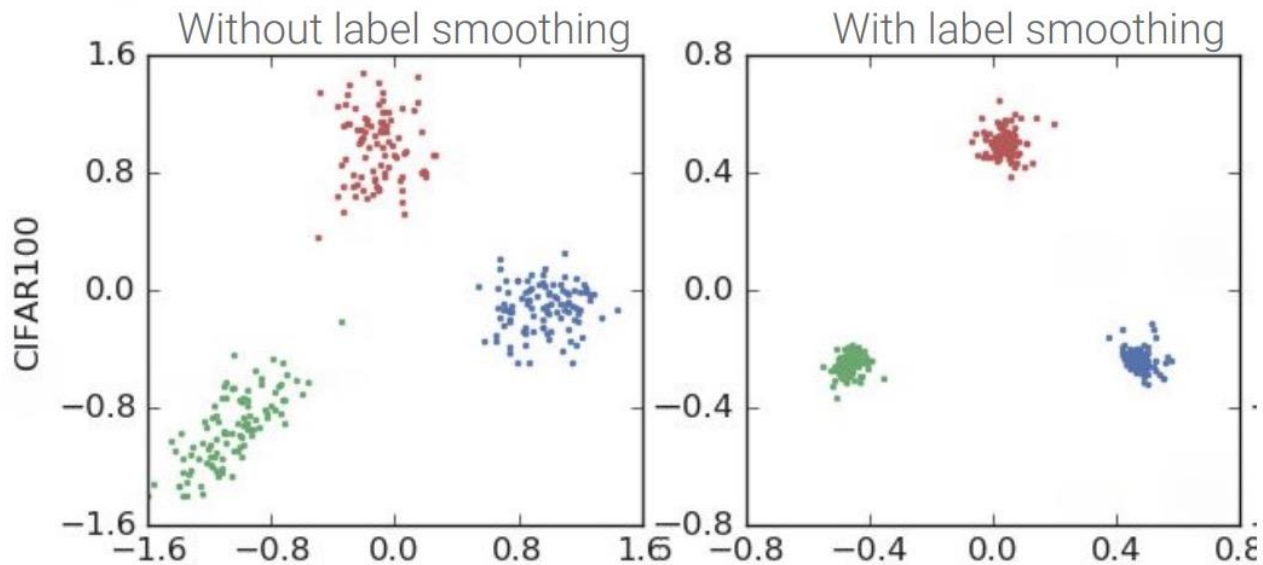
Label smoothing replaces one-hot encoded label vector y with a mixture of y and the uniform distribution

$$y_k^{LS} = y_k(1 - \alpha) + \alpha/K$$

May help generalisation! It depends ...



Label smoothing



With label smoothing, activation is close to prototype of correct class and equally distant to prototypes of all remaining classes.

encouraging smoother decision boundaries

Label smoothing in PyTorch

```
confidence = 1.0 - smoothing  
smooth_value = smoothing / num_classes
```

```
# Apply smoothing  
smoothed_labels = one_hot * confidence + smooth_value
```

If label smoothing = 0.0, what happens?

Summary

Which regularisation technique adds randomness to activations? →

Which regularisation increases training data variety? →

Which regularisation softens target distributions? →

What happens if dropout is not disabled during evaluation? →

Summary

Which regularisation technique adds randomness to activations? → Dropout

Which regularisation increases training data variety? →

Which regularisation softens target distributions? →

What happens if dropout is not disabled during evaluation? →

Summary

Which regularisation technique adds randomness to activations? → Dropout

Which regularisation increases training data variety? → Data Augmentation

Which regularisation softens target distributions? →

What happens if dropout is not disabled during evaluation? →

Summary

Which regularisation technique adds randomness to activations? → Dropout

Which regularisation increases training data variety? → Data Augmentation

Which regularisation softens target distributions? → Label Smoothing

What happens if dropout is not disabled during evaluation? →

Summary

Which regularisation technique adds randomness to activations? → Dropout

Which regularisation increases training data variety? → Data Augmentation

Which regularisation softens target distributions? → Label Smoothing

What happens if dropout is not disabled during evaluation? → Unstable predictions

Next week

Other regularisations (batch normalisation, early stopping...)

A whole training pipeline of DNNs with regularisations